

Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Інститут телекомунікацій, радіоелектроніки та електронної техніки



Лабораторна робота
З дисципліни
“Об'єктно-орієнтоване програмування, частина 2”

Виконав: ст.гр IX-22

Горячий Ростислав

Прийняв:

Гордійчук-Бублівська О.В

Львів – 2026

Тема роботи:

Візуалізація даних за допомогою бібліотеки Seaborn

Мета роботи:

Навчитися працювати з бібліотекою Seaborn для візуалізації даних.

Ознайомитися з основними типами графіків, які надає бібліотека, навчитися застосовувати їх для аналізу даних.

- Створення теплової карти (Heatmap)
- Створіть теплову карту для аналізу кількості вакансій за рівнем досвіду (Experience Level) та галуззю (Industry).

Підказка: Можна припустити, що кількість вакансій рівна кількості рядків у датафреймі. Тоді скористатися `crosstab()`, що підрахує кількість рядків у датафреймі `df`, які одночасно мають певний рівень досвіду та відносяться до певної галузі:

```
pivot_table = pd.crosstab(df['Experience'], df['Industry'])
```

Або ж використати `groupby()`

- Зробіть висновок про те, де спостерігається найбільша кількість вакансій.
- Створення точкової діаграми (Scatterplot)
- Відобразіть залежність зарплати (Average Salary) від року публікації вакансії (Year, треба витягнути з Date Posted). Точки на графіку пофарбуйте за рівнем досвіду (`hue='Experience Level'`).
- Зробіть висновки коли спостерігається тенденція до зростання зарплат, для вакансій з яким рівнем досвіду.
- Парні графіки (Pairplot)
- Створіть парні графіки для аналізу взаємозв'язків між зарплатою, роком та рівнем досвіду.
- Зробіть висновки з отриманих залежностей та в чому полягає відмінність точкової діаграми від парних графіків.
- Зробіть загальні висновки про роботу

et.py > ...

```
1 import pandas as pd
2 import seaborn as sns
3 import matplotlib.pyplot as plt
4
5 class CSV:
6     def __init__(self, file: str):
7         self.file=file
8         self.dataset=None
9     def open_csv(self):
10         self.dataset=pd.read_csv(self.file)
11         return self.dataset
12
13 class DataFrame:
14     def __init__(self, df):
15         self.df=df
16     def add_average_salary(self):
17         min_s = self.df['Salary Range'].str.extract(r'E(\d+,?\d*) -')[0]
18         min_s = min_s.str.replace(',','').astype(int)
19         max_s = self.df['Salary Range'].str.extract(r'E(\d+,?\d*)')[0]
20         max_s = max_s.str.replace(',','').astype(int)
21         avg_salary = (max_s + min_s) / 2
22         self.df["Average Salary"] = avg_salary
23         return self.df
24     def add_year_col(self):
25         def get_year(date):
26             return pd.to_datetime(date, format='%m/%d/%Y').year
27         self.df['Year'] = self.df['Date Posted'].apply(get_year)
28         return self.df
29
30 class Barplot:
31     def __init__(self, data: pd.DataFrame, x_param: str, y_param: str):
32         self.data=data
33         self.x_param=x_param
34         self.y_param=y_param
35     def plot(self, title ,x_label, y_label):
36         try:
37             salary_experience=self.data[[self.x_param, self.y_param]]
38             plt.figure(figsize=(10, 6))
39             sns.barplot(x=self.x_param, y=self.y_param, data=salary_experience, palette='viridis')
40             plt.title(title)
41             plt.xlabel(x_label)
42             plt.ylabel(y_label)
43             plt.show()
44         except KeyError:
45             print("unknown key")
46
47 class Boxplot:
48     def __init__(self, data: pd.DataFrame, x_param: str, y_param: str):
49         self.data=data
50         self.x_param=x_param
```

```

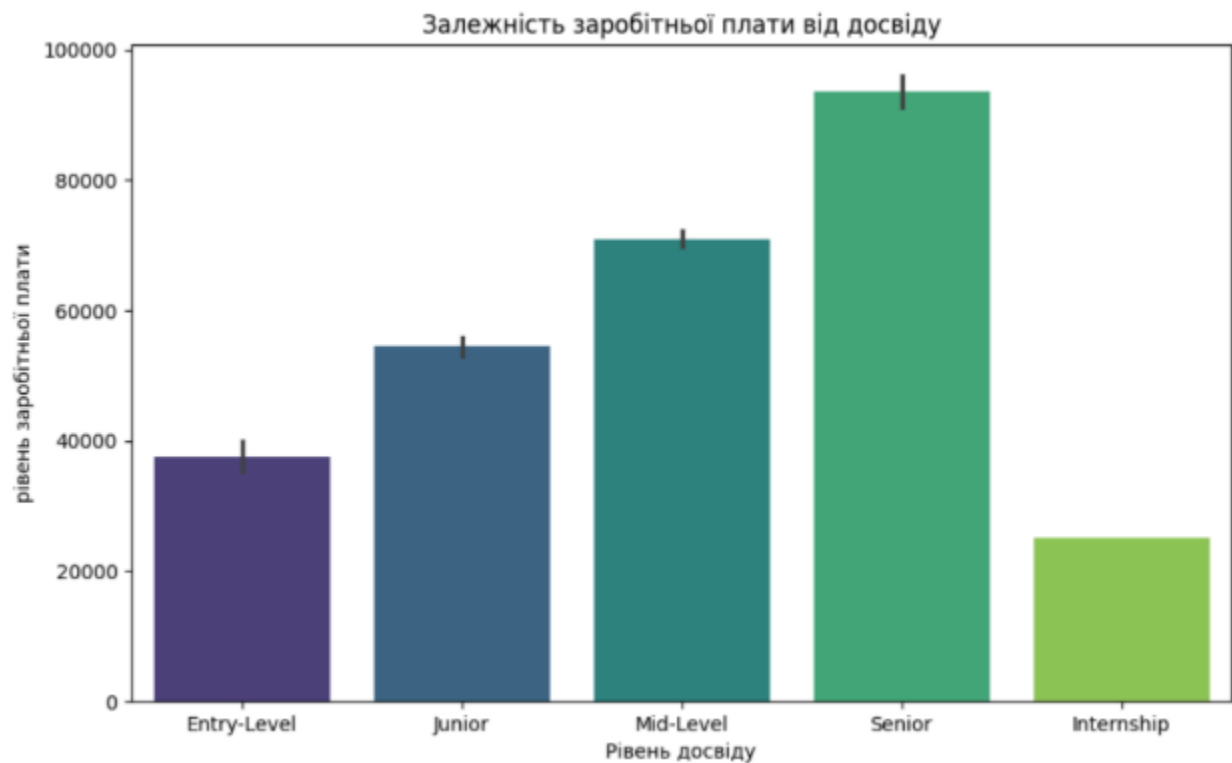
49     self.y_param=y_param
50     def plot(self, title ,x_label, y_label):
51         try:
52             salary_industry = self.data[[self.x_param, self.y_param]]
53             plt.figure(figsize=(12, 6))
54             plt.xticks(rotation=45, ha='right')
55             plt.tight_layout()
56             sns.boxplot(x=self.x_param, y=self.y_param, data=salary_industry, palette='viridis')
57             plt.title(title)
58             plt.xlabel(x_label)
59             plt.ylabel(y_label)
60             plt.show()
61         except KeyError:
62             print("unknown key")
63         except ValueError:
64             print("value error")
65     class Heatmap:
66         def __init__(self, data: pd.DataFrame, x_param: str, y_param: str):
67             self.data = data
68             self.x_param = x_param
69             self.y_param = y_param
70
71         def plot(self, title):
72             try:
73                 plt.figure(figsize=(10,6))
74                 pivot_table = pd.crosstab(self.data[self.x_param], self.data[self.y_param])
75                 sns.heatmap(pivot_table)
76                 plt.title(title)
77                 plt.show()
78             except KeyError:
79                 print("unknown key")
80             except ValueError:
81                 print("value error")
82     class Scatterplot:
83         def __init__(self, data: pd.DataFrame, x_param: str, y_param: str, hue: str):
84             self.data = data
85             self.x_param = x_param
86             self.y_param = y_param
87             self.hue=hue
88         def plot(self, title, x_label, y_label):
89             try:
90                 plt.figure(figsize=(10, 6))
91                 sns.scatterplot(x=self.x_param, y=self.y_param, hue=self.hue, data=self.data)
92                 plt.title(title)
93                 plt.xlabel(x_label)
94                 plt.ylabel(y_label)
95                 plt.show()
96             except KeyError:

```

```

96             except KeyError:
97                 print("unknown key")
98             except ValueError:
99                 print("value error")
100
101     class Pairplot:
102         def __init__(self, data: pd.DataFrame, param_list: list):
103             self.data = data
104             self.param_list=param_list
105         def plot(self, title):
106             try:
107                 sns.pairplot(self.data[self.param_list], diag_kind='kde', palette='bright')
108                 plt.suptitle(title, y=1.02)
109                 plt.show()
110             except KeyError:
111                 print("unknown key")
112             except ValueError:
113                 print("value error")
114
115     ds=CSV("Job opportunities.csv").open_csv()
116
117     dataframe=DataFrame(ds)
118     df_with_avg_salary=dataframe.add_average_salary()
119     df=dataframe.add_year_col()
120
121     Barplot(df, "Experience Level", "Average Salary").plot("Залежність заробітної плати від досвіду",
122                                                         "Рівень досвіду", "рівень заробітної плати")
123     Boxplot(df, "Industry", "Average Salary"). plot("Діаграма розмаху для відображення розподілу зарплат за галузями",
124                                                         "Галузь", "Середня заробітня плата")
125     Heatmap(df, "Industry", "Experience Level").plot("Теплова карта для аналізу кількості вакансій за рівнем досвіду та галуззю")
126     Scatterplot(df, "Year", "Average Salary", "Experience Level").plot("залежність зарплати від року публікації",
127                                                         "Рік публікації", "Середня зарплата")
128     Pairplot(df, ["Average Salary", "Year", "Experience Level"]).plot("Аналіз взаємозв'язків між зарплатою, роком та рівнем досвіду")

```



Висновок:

Під час виконання лабораторної роботи я опанував інструментарій бібліотеки **Seaborn** для візуалізації даних. Зокрема, ознайомився з основними типами графіків, які надає ця бібліотека, та навчився застосовувати їх для проведення аналітичних досліджень і наочного представлення результатів